Assignment 4

1. ARIES Recovery Algorithm

(a)

- ARIES stands for Algorithms for Recovery and Isolation Exploiting Semantics.
- It is a recovery algorithm used in DBMS to ensure data consistency after a crash.
- It follows the Write-Ahead Logging (WAL) protocol.

Analysis

- Begins from the last checkpoint in the log.
- Identifies:
    - Active transactions at crash time.
    - Dirty pages (modified but not written to disk).
- Creates:
    - Transaction Table (TT)
    - Dirty Page Table (DPT)

Log Usage

- Scans log records to determine system state before crash.

Redo

- Re-applies all updates from the log.
- Ensures all committed changes are reflected in the database.

Key Idea

- “Repeat history” (redo everything necessary).

Log Usage

- Uses update log records like T1: X = 500.

Undo

- Reverses changes made by uncommitted transactions.
- Uses Compensation Log Records (CLR).

Log Usage

- Traverses logs backward to undo incomplete transactions.

(b)

Log-Based Recovery Analysis

Given Log

- Start T1
- T1: X = 500
- Start T2
- T2: Y = 300
- T1 Commit
- Crash

Transaction Classification

- T1 → Committed → Winner
- T2 → Not committed → Loser

Recovery Actions

- Redo: T1
- Undo: T2
- Ignore: None

Justification

- Committed transactions must be preserved (Redo).
- Uncommitted transactions must be rolled back (Undo).

2.

(a) Timestamp Ordering Protocol (TSO)

- A concurrency control protocol based on timestamps.
- Each transaction is assigned a unique timestamp.

Key Variables

- Read_TS(X): Largest timestamp of any transaction that read X.
- Write_TS(X): Largest timestamp of any transaction that wrote X.

Rules

Read Operation

- Allowed if:
  TS(T) ≥ Write_TS(X)
- Otherwise → Transaction is rolled back.

Write Operation

- Allowed if:
  - TS(T) ≥ Read_TS(X)
  - TS(T) ≥ Write_TS(X)
  - Otherwise → Transaction is rolled back.

(b) Schedule Execution

Given Timestamps

- TS(T1) = 8
- TS(T2) = 12
- TS(T3) = 20

Execution Steps

Step 1: T2 → R(X)

- Allowed
- Read_TS(X) = 12

Step 2: T1 → W(X)

- TS(T1) < Read_TS(X)
- T1 is rolled back

Step 3: T3 → R(X)

- Allowed
- Read_TS(X) = 20

Step 4: T2 → W(X)

- TS(T2) < Read_TS(X)
- T2 is rolled back

Final Result

- Rolled Back: T1, T2
- Executed Successfully: T3

3. NoSQL Databases

Schema Flexibility

- Schema-less or dynamic schema.
- Allows storing unstructured or semi-structured data.

Scalability

- Horizontally scalable (supports sharding).
- Designed for distributed environments.

ACID Support

- Typically follows BASE model:
    - Basically Available
    - Soft state
    - Eventually consistent
- Some NoSQL systems provide partial ACID support.

Common Applications

- Big data systems
- Real-time analytics
- Social media platforms
- IoT applications
- Content management systems

4.

(i) CAP Theorem

- A distributed system can guarantee only two of the following three properties.

Components

Consistency (C)

- All nodes have the same data at the same time.

Availability (A)

- Every request receives a response.

Partition Tolerance (P)

- System continues to function despite network failures.

System Classification

MongoDB

- Follows CP (Consistency + Partition Tolerance)
- May sacrifice availability during network partition.

RDBMS

- Follows CA (Consistency + Availability)
- Assumes no network partition.

(ii) Graph Database Use-Cases

When to Use

- When relationships between data are complex and important.

Applications

Social Networks

- Managing friend relationships

Recommendation Systems

- Product or content suggestions

Fraud Detection

- Detecting unusual patterns in transactions

Knowledge Graphs

- Representing semantic relationships

Example System

- Neo4j

5.

(i) Star Schema Design

Fact Table: Sales

- Sale_ID (Primary Key)
- Product_ID (Foreign Key)
- Customer_ID (Foreign Key)
- Time_ID (Foreign Key)
- Amount

Dimension Tables

Product Table

- Product_ID (Primary Key)
- Product_Name
- Category

Customer Table

- Customer_ID (Primary Key)
- Name
- Location

Time Table

- Time_ID (Primary Key)
- Date
- Month
- Year

Structure Explanation

- Fact table is central.
- Dimension tables surround it.
- Foreign keys connect fact to dimensions.

(ii) OLAP Operations

Roll-Up

- Aggregates data (e.g., day → month → year).

Drill-Down

- Breaks data into finer detail.

Slice

- Selects a single dimension value.

Dice

- Filters multiple dimensions.

Pivot

- Changes orientation of data view.

(iii) Apriori Algorithm

Working

- Generates candidate itemsets.
- Prunes infrequent ones using support.
- Repeats until no new itemsets are found.

Frequent Itemsets (Min Support = 50%)

1-Itemsets

- Milk (3)
- Bread (3)
- Diaper (3)
- Beer (2)

2-Itemsets

- Milk, Bread (2)
- Milk, Diaper (2)
- Bread, Diaper (2)
- Diaper, Beer (2)

3-Itemsets

- None meet threshold

Association Rules (Confidence ≥ 50%)

- Milk → Bread (66%)
- Bread → Milk (66%)
- Diaper → Beer (66%)
- Beer → Diaper (100%)

(iv) FP-Growth Algorithm

Working

- Builds an FP-tree from transactions.
- Avoids candidate generation.
- Extracts frequent patterns efficiently.

Result

- Produces same frequent itemsets as Apriori.
- Faster for large datasets.

6.

(a) Two-Phase Locking (2PL)

- A concurrency control method ensuring serializability.

Phases

Growing Phase

- Transactions acquire locks.
- No locks are released.

Shrinking Phase

- Locks are released.
- No new locks are acquired.

Variants

Basic 2PL

- Locks can be released anytime after use.

- Ensures serializability.

Strict 2PL

- Locks released only after commit.
- Prevents cascading rollbacks.

Rigorous 2PL

- All locks released only after commit.
- Strongest consistency guarantee.

(b) Timestamp Ordering Execution

Given

- TS(T1) = 3
- TS(T2) = 7
- TS(T3) = 12

Steps

Step 1: T1 → R(A)

- Allowed
- Read_TS(A) = 3

Step 2: T3 → W(A)

- Allowed
- Write_TS(A) = 12

Step 3: T2 → R(A)

- TS(T2) < Write_TS(A)
- T2 is aborted

Step 4: T1 → W(B)

- Allowed (independent operation)

Result

- Aborted: T2
- Successful: T1, T3